

Esercizi SQL

Simone Melzi
2023/2024

ESERCIZIO ESAME

Data la seguente base di dati:

STUDENTE(matricola, CF, cognome, nome, data_nascita)

INSEGNAMENTO(codice, nome, crediti)

ESAME(studente, insegnamento, data, voto, lode)

Si formulino le seguenti query SQL:

1. Il codice e il nome degli insegnamenti che valgono il maggior numero di crediti

ESERCIZIO ESAME

Data la seguente base di dati:

STUDENTE(matricola, CF, cognome, nome, data_nascita)

INSEGNAMENTO(codice, nome, crediti)

ESAME(studente, insegnamento, data, voto, lode)

Si formulino le seguenti query SQL:

1. Il codice e il nome degli insegnamenti che valgono il maggior numero di crediti

```
SELECT I.codice, I.nome
```

```
FROM INSEGNAMENTO AS I
```

```
WHERE I.crediti = (SELECT MAX(I.crediti)
```

```
FROM INSEGNAMENTO AS I)
```

ESERCIZIO ESAME

Data la seguente base di dati:

STUDENTE(matricola, CF, cognome, nome, data_nascita)

INSEGNAMENTO(codice, nome, crediti)

ESAME(studente, insegnamento, data, voto, lode)

Si formulino le seguenti query SQL:

1. Il codice e il nome degli insegnamenti che valgono il maggior numero di crediti

```
SELECT I.codice, I.nome  
FROM INSEGNAMENTO AS I  
WHERE I.crediti = (SELECT MAX(I.crediti)  
                  FROM INSEGNAMENTO AS I)
```

2. Matricola e media voto degli studenti nati nel 2002.

ESERCIZIO ESAME

Data la seguente base di dati:

STUDENTE(matricola, CF, cognome, nome, data_nascita)

INSEGNAMENTO(codice, nome, crediti)

ESAME(studente, insegnamento, data, voto, lode)

Si formulino le seguenti query SQL:

1. Il codice e il nome degli insegnamenti che valgono il maggior numero di crediti

```
SELECT I.codice, I.nome
FROM INSEGNAMENTO AS I
WHERE I.crediti = (SELECT MAX(I.crediti)
                  FROM INSEGNAMENTO AS I)
```

2. Matricola e media voto degli studenti nati nel 2002.

```
SELECT S.matricola, AVG(E.voto)
FROM STUDENTE AS S JOIN ESAME AS E ON E.studente = S.matricola
WHERE S.anno_nascita = 2002
GROUP BY S.matricola
```

ESERCIZIO ESAME

Data la seguente base di dati:

STUDENTE(matricola, CF, cognome, nome, data_nascita)

INSEGNAMENTO(codice, nome, crediti)

ESAME(studente, insegnamento, data, voto, lode)

Si formulino le seguenti query SQL:

1. Il codice e il nome degli insegnamenti che valgono il maggior numero di crediti

```
SELECT I.codice, I.nome
FROM INSEGNAMENTO AS I
WHERE I.crediti = (SELECT MAX(I.crediti)
                  FROM INSEGNAMENTO AS I)
```

2. Matricola e media voto degli studenti nati nel 2002.

```
SELECT S.matricola, AVG(E.voto)
FROM STUDENTE AS S JOIN ESAME AS E ON E.studente = S.matricola
WHERE S.anno_nascita = 2002
GROUP BY S.matricola
```

3. La matricola, il nome, il cognome dello studente che ha preso il massimo voto nell'esame dell'insegnamento di codice 'BDD01'.

ESERCIZIO ESAME

Data la seguente base di dati:

STUDENTE(matricola, CF, cognome, nome, data_nascita)

INSEGNAMENTO(codice, nome, crediti)

ESAME(studente, insegnamento, data, voto, lode)

Si formulino le seguenti query SQL:

1. Il codice e il nome degli insegnamenti che valgono il maggior numero di crediti

```
SELECT I.codice, I.nome  
FROM INSEGNAMENTO AS I  
WHERE I.crediti = (SELECT MAX(I.crediti)  
                  FROM INSEGNAMENTO AS I)
```

2. Matricola e media voto degli studenti nati nel 2002.

```
SELECT S.matricola, AVG(E.voto)  
FROM STUDENTE AS S JOIN ESAME AS E ON E.studente = S.matricola  
WHERE S.anno_nascita = 2002  
GROUP BY S.matricola
```

3. La matricola, il nome, il cognome dello studente che ha preso il massimo voto nell'esame dell'insegnamento di codice 'BDD01'.

```
SELECT S.matricola, S.nome, S.cognome  
FROM STUDENTE AS S JOIN ESAME AS E ON S.matricola = E.studente  
WHERE E.insegnamento = 'BDD01' AND 'Voto' = (SELECT MAX (voto)  
                                                FROM ESAME WHERE E.insegnamento = 'BDD01')
```

Esercizio A

Data la seguente base di dati

Aeroporto(Città, Nazione, NumPiste)

Volo(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, Aereo)

Aereo(TipoAereo, NumPasseggeri, QtaMerci)

Interrogazioni SQL:

- 1) Selezionare tutti gli ID dei voli che utilizzano l'AIRBUS 400.
- 2) Selezionare gli aeroporti che hanno il maggior numero di piste.
- 3) Selezionare le tipologie di aerei che hanno viaggiato in italia.
- 4) Selezionare le tipologie di aerei che hanno viaggiato maggiormente.
- 5) Selezionare le tipologie di aerei che hanno effettuato più di 100 voli

Esercizio A.1 Soluzione

Aeroporto(Città, Nazione, NumPiste)

Volo(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, Aereo)

Aereo(TipoAereo, NumPasseggeri, QtaMerci)

1. Selezionare tutti gli ID dei voli che utilizzano l'AIRBUS 400.

```
SELECT V.IdVolo  
FROM Volo AS V  
WHERE V.Aereo = 'AIRBUS 400'
```

Esercizio A.2 Soluzione

Aeroporto(Città, Nazione, NumPiste)

Volo(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, Aereo)

Aereo(TipoAereo, NumPasseggeri, QtaMerci)

2. Selezionare gli aeroporti che hanno il maggior numero di piste.

```
SELECT A.Città
FROM Aeroporto AS A
WHERE A.NumPiste = (SELECT MAX(A1.NumPiste)
                   FROM Aeroporto AS A1)
```

Esercizio A.3 Soluzione

Aeroporto(Città, Nazione, NumPiste)

Volo(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, Aereo)

Aereo(TipoAereo, NumPasseggeri, QtaMerci)

3. Selezionare le tipologie di aerei che hanno viaggiato in italia.

```
SELECT DISTINCT V.Aereo
FROM (Aeroporto AS A1 JOIN Volo AS V ON V.CittàPart = A1.Città)
     JOIN Aeroporto AS A2 ON V.CittàArr = A2.Città
WHERE A1.Nazione = 'Italia' AND A2.Nazione = 'Italia'
```

Esercizio A.4 Soluzione

Aeroporto(Città, Nazione, NumPiste)

Volo(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, Aereo)

Aereo(TipoAereo, NumPasseggeri, QtaMerci)

4. Selezionare le tipologie di aerei che hanno viaggiato maggiormente.

```
CREATE VIEW NumVoliTipo AS
```

```
SELECT V.Aereo, COUNT(*) AS NumVoli
```

```
FROM Volo AS V
```

```
GROUP BY V.Aereo
```

```
SELECT V.Aereo COUNT(*) AS NumVoli
```

```
FROM Volo AS V
```

```
GROUP BY V.Aereo
```

```
HAVING NumVoli = (SELECT MAX(N.NumVoli)
```

```
FROM NumVoliTipo AS N)
```

Esercizio A.5 Soluzione

Aeroporto(Città, Nazione, NumPiste)

Volo(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, Aereo)

Aereo(TipoAereo, NumPasseggeri, QtaMerci)

5. Selezionare le tipologie di aerei che hanno effettuato più di 100 voli

```
SELECT V.Aereo, COUNT(*) AS NumVoli  
FROM Volo AS V  
GROUP BY V.Aereo  
HAVING NumVoli > 100
```

Esercizio B

Data la seguente base di dati che descrive un campionato invernale di sci maschile

LUOGOGARA(nome, altitudine, nazione)

GARA (specialità, luogo, dataGara, lunghezza, dislivello, premiovincitore)

POSIZIONEGARA (idatleta, dataGara, numeroPettorina, specialità, luogo, posizionearrivo)

ATLETA (idatleta, nome, cognome, nazione, datadinascita)

Rispondere alle seguenti interrogazioni in SQL:

1. Trovare per ogni nazione l'atleta che ha vinto il maggior numero di gare (assumendo che i valori di posizione arrivo siano specificati da interi).
2. Trovare le nazioni che hanno almeno un'atleta che ha vinto almeno una gara in patria.

Esercizio B.1 Soluzione

LUOGOGARA(nome, altitudine, nazione)

GARA (specialità, luogo, dataGara, lunghezza, dislivello, premiovincitore)

POSIZIONEGARA (idatleta, dataGara, numeroPettorina, specialità, luogo, posizionearrivo)

ATLETA (idatleta, nome, cognome, nazione, datadinascita)

1. Trovare per ogni nazione l'atleta che ha vinto il maggior numero di gare (assumendo che i valori di posizionearrivo siano specificati da interi).

Esercizio B.1 Soluzione

LUOGOGARA(nome, altitudine, nazione)

GARA (specialità, luogo, dataGara, lunghezza, dislivello, premiovincitore)

POSIZIONEGARA (idatleta, dataGara, numeroPettorina, specialità, luogo, posizionearrivo)

ATLETA (idatleta, nome, cognome, nazione, datadinascita)

1. Trovare per ogni nazione l'atleta che ha vinto il maggior numero di gare (assumendo che i valori di posizionearrivo siano specificati da interi).

```
SELECT A1.nazione, A1.nome, A1.cognome
FROM ATLETA AS A1 JOIN POSIZIONEGARA AS P1 ON A1.idatleta = P1.idatleta
WHERE posizionearrivo = 1
GROUP BY A1.nazione, A1.nome, A1.cognome
HAVING COUNT(*) >= ALL (SELECT COUNT(*)
FROM ATLETA AS A2 JOIN POSIZIONEGARA AS P2 ON A2.idatleta=P2.idatleta
WHERE posizionearrivo = 1 AND A1.nazione = A2.nazione
GROUP BY A2.idatleta)
```

Esercizio B.2 Soluzione

LUOGOGARA(nome, altitudine, nazione)

GARA (specialità, luogo, dataGara, lunghezza, dislivello, premiovincitore)

POSIZIONEGARA (idatleta, dataGara, numeroPettorina, specialità, luogo, posizionearrivo)

ATLETA (idatleta, nome, cognome, nazione, datadinascita)

2. Trovare le nazioni che hanno almeno un'atleta che ha vinto almeno una gara in patria.

```
SELECT DISTINCT nazione
FROM ATLETA AS A JOIN POSIZIONEGARA AS P ON A.idatleta =
P.idatleta JOIN LUOGOGARA AS L ON L.nome = P.luogo
WHERE P.posizionearrivo = 1 AND A.nazione = L.nazione
```

Esercizio C

Data la seguente base di dati

CLIENTE(CodiceFiscale, Nome, Cognome, Nazionalità, Telefono, e-mail)

STANZA(CodiceStanza, Albergo, Tipo, MetriQuadri, Locali, Piano)

SOGGIORNO(CodiceStanza, Albergo, DataInizio, DataFine, Agente, CFCliente, CostoGiornaliero)

ALBERGO(CodiceAlbergo, NumeroDiStelle, Indirizzo, Città, Stato)

AGENTE(CodiceAgente, Nome, Cognome, DataDiNascita).

Rispondere alle seguenti interrogazioni in SQL:

1. Trovare, specificando nome cognome e codice fiscale, i clienti omonimi (stesso nome e cognome e codice fiscale diverso) che abbiano soggiornato in una qualunque stanza di albergo nella medesima città.
2. Trovare l'agente (codice, nome, cognome) che ha stipulato più soggiorni per l'anno 2009 (cominciati nel 2009) in alberghi con più di tre stelle.

Esercizio C.1 Soluzione

CLIENTE(CodiceFiscale, Nome, Cognome, Nazionalità, Telefono, e-mail)

STANZA(CodiceStanza, Albergo, Tipo, MetriQuadri, Locali, Piano)

SOGGIORNO(CodiceStanza, Albergo, DataInizio, DataFine, Agente, CFCliente, CostoGiornaliero)

ALBERGO(CodiceAlbergo, NumeroDiStelle, Indirizzo, Città, Stato)

AGENTE(CodiceAgente, Nome, Cognome, DataDiNascita).

1. Trovare, specificando nome cognome e codice fiscale, i clienti omonimi (stesso nome e cognome e codice fiscale diverso) che abbiano soggiornato in una qualunque stanza di albergo nella medesima città.

Esercizio C.1 Soluzione

CLIENTE(CodiceFiscale, Nome, Cognome, Nazionalità, Telefono, e-mail)

STANZA(CodiceStanza, Albergo, Tipo, MetriQuadri, Locali, Piano)

SOGGIORNO(CodiceStanza, Albergo, DataInizio, DataFine, Agente, CFCliente, CostoGiornaliero)

ALBERGO(CodiceAlbergo, NumeroDiStelle, Indirizzo, Città, Stato)

AGENTE(CodiceAgente, Nome, Cognome, DataDiNascita).

1. Trovare, specificando nome cognome e codice fiscale, i clienti omonimi (stesso nome e cognome e codice fiscale diverso) che abbiano soggiornato in una qualunque stanza di albergo nella medesima città.

```
SELECT Nome, Cognome, CodiceFiscale
FROM (CLIENTE AS C JOIN SOGGIORNO ON C.CodiceFiscale = CFCliente)
     JOIN ALBERGO AS A ON A.CodiceAlbergo = Albergo
WHERE EXISTS (SELECT *
              FROM (CLIENTE AS C1 JOIN SOGGIORNO AS S1 ON
                    C1.CodiceFiscale = S1.CFCliente) JOIN Albergo A1 ON
                    A1.CodiceAlbergo = S1.Albergo
              WHERE C1.nome = C.nome AND C1.cognome = C.cognome AND
                    C1.CodiceFiscale <> C.CodiceFiscale AND A.Città=A1.Città)
```

Esercizio C.2 Soluzione

CLIENTE(CodiceFiscale, Nome, Cognome, Nazionalità, Telefono, e-mail)

STANZA(CodiceStanza, Albergo, Tipo, MetriQuadri, Locali, Piano)

SOGGIORNO(CodiceStanza, Albergo, DataInizio, DataFine, Agente, CFCliente, CostoGiornaliero)

ALBERGO(CodiceAlbergo, NumeroDiStelle, Indirizzo, Città, Stato)

AGENTE(CodiceAgente, Nome, Cognome, DataDiNascita).

2. Trovare l'agente (codice, nome, cognome) che ha stipulato più soggiorni per l'anno 2009 (cominciati nel 2009) in alberghi con più di tre stelle.

Esercizio C.2 Soluzione

CLIENTE(CodiceFiscale, Nome, Cognome, Nazionalità, Telefono, e-mail)

STANZA(CodiceStanza, Albergo, Tipo, MetriQuadri, Locali, Piano)

SOGGIORNO(CodiceStanza, Albergo, DataInizio, DataFine, Agente, CFCliente, CostoGiornaliero)

ALBERGO(CodiceAlbergo, NumeroDiStelle, Indirizzo, Città, Stato)

AGENTE(CodiceAgente, Nome, Cognome, DataDiNascita).

2. Trovare l'agente (codice, nome, cognome) che ha stipulato più soggiorni per l'anno 2009 (cominciati nel 2009) in alberghi con più di tre stelle.

```
SELECT CodiceAgente, Nome, Cognome
FROM AGENTE JOIN SOGGIORNO ON CodiceAgente = Agente
      JOIN ALBERGO ON Albergo = CodiceAlbergo
WHERE DataInizio >= 1-1-2009 AND DataInizio < 1-1-2010 AND NumeroDiStelle > 3
GROUP BY CodiceAgente, Nome, Cognome
HAVING COUNT(*) >= ALL (SELECT COUNT (*)
                        FROM AGENTE JOIN SOGGIORNO ON
                                CodiceAgente = Agente JOIN ALBERGO ON
                                        Albergo = CodiceAlbergo
                        WHERE DataInizio >= 1-1-2009 AND DataInizio < 1-1-2010
                        AND NumeroDiStelle > 3
                        GROUP BY CodiceAgente)
```

Esercizio D

Si consideri il seguente schema che rappresenta una base di dati impiegata in una catena di librerie distribuite su tutto il territorio nazionale. In particolare oltre ai dati sulle singole librerie, sono archiviati i dati del personale e dei libri in vendita. Si osservi che possono esistere diverse versioni dello stesso libro (autore e titolo), edite da case editrici diverse. Infine attraverso la relazione Catalogo è possibile conoscere in quali librerie una determinata versione di un libro sia disponibile per essere venduta, il numero di copie a disposizione, ed il costo.

LIBRERIA(idLibreria, nome, indirizzo, città, oraApertura, oraChiusura, turnoChiusura)

PERSONALE(idPersona, cognome, nome, idLibreria, reparto, turno)

CATALOGO(idLibro, idLibreria, reparto, numerocopie, costo)

LIBRO(idLibro, titolo, autori, casaEditrice, annoPubblicazione)

1. Il nome e la città delle libreria dove siano presenti almeno 10 copie del libro “Delitto e castigo”, tenendo conto che possono esistere più versioni del libro poichè stampato da più case editrici.
2. La libreria di Milano con il numero maggiore di libri in vendita (contando quindi tutte le copie di ogni libro).

Esercizio D.1 Soluzione

LIBRERIA(idLibreria, nome, indirizzo, città, oraApertura, oraChiusura, turnoChiusura)

PERSONALE(idPersona, cognome, nome, idLibreria, reparto, turno)

CATALOGO(idLibro, idLibreria, reparto, numerocopie, costo)

LIBRO(idLibro, titolo, autori, casaEditrice, annoPubblicazione)

1. Il nome e la città delle libreria dove siano presenti almeno 10 copie del libro “Delitto e castigo”, tenendo conto che possono esistere più versioni del libro poichè stampato da più case editrici.

Esercizio D.1 Soluzione

LIBRERIA(idLibreria, nome, indirizzo, città, oraApertura, oraChiusura, turnoChiusura)

PERSONALE(idPersona, cognome, nome, idLibreria, reparto, turno)

CATALOGO(idLibro, idLibreria, reparto, numerocopie, costo)

LIBRO(idLibro, titolo, autori, casaEditrice, annoPubblicazione)

1. Il nome e la città delle libreria dove siano presenti almeno 10 copie del libro “Delitto e castigo”, tenendo conto che possono esistere più versioni del libro poichè stampato da più case editrici.

```
CREATE VIEW NumCopieLibreria(idLibreria, totalecopie) AS
```

```
SELECT idLibreria, SUM(numerocopie)
```

```
FROM CATALOGO JOIN LIBRO ON CATALOGO.idLibro = LIBRO.idLibro
```

```
WHERE titolo = 'Delitto e castigo'
```

```
GROUP BY idLibreria
```

```
SELECT nome, città
```

```
FROM LIBRERIA AS L JOIN NumCopieLibreria AS N ON L.idLibreria = N.idLibreria
```

```
WHERE totalecopie >= 10
```

Esercizio D.2 Soluzione

LIBRERIA(idLibreria, nome, indirizzo, città, oraApertura, oraChiusura, turnoChiusura)

PERSONALE(idPersona, cognome, nome, idLibreria, reparto, turno)

CATALOGO(idLibro, idLibreria, reparto, numerocopie, costo)

LIBRO(idLibro, titolo, autori, casaEditrice, annoPubblicazione)

2. La libreria di Milano con il numero maggiore di libri in vendita (contando quindi tutte le copie di ogni libro).

Esercizio D.2 Soluzione

LIBRERIA(idLibreria, nome, indirizzo, città, oraApertura, oraChiusura, turnoChiusura)

PERSONALE(idPersona, cognome, nome, idLibreria, reparto, turno)

CATALOGO(idLibro, idLibreria, reparto, numerocopie, costo)

LIBRO(idLibro, titolo, autori, casaEditrice, annoPubblicazione)

2. La libreria di Milano con il numero maggiore di libri in vendita (contando quindi tutte le copie di ogni libro).

```
SELECT idLibreria, nome
FROM LIBRERIA JOIN CATALOGO ON CATALOGO.idLibreria = LIBRERIA.idLibreria
WHERE città=Milano
GROUP BY idLibreria, nome
HAVING SUM(numerocopie) >= ALL( SELECT SUM(numerocopie)
                                FROM LIBRERIA JOIN CATALOGO ON
                                    CATALOGO.idLibreria = LIBRERIA.idLibreria
                                WHERE città = 'Milano'
                                GROUP BY idLibreria)
```

Esercizio E

Si consideri la seguente Base di Dati:

CINEMA(codice,nome, indirizzo,codice_comune)

COMUNE(codice,nome,provincia,regione,numero_abitanti)

FILM(codice,titolo,anno,nazionalita,tecnica_animazione)

PROIETTATO_IN(codice_film,codice_sala,data_ora,lingua)

SALA(codice,nome,capienza,abilitata_3d,codice_cinema)

Si formulino le seguenti query in SQL

1. Selezionare tutti i film di tecnica animazione «computer graphics» proiettati nel 2016
2. Selezionare la capienza media delle sale dei cinema che hanno più di una sala
3. Selezionare i cinema e nome della città che hanno il maggior numero di posti a sedere.

Esercizio E.1 Soluzione

CINEMA(codice,nome, indirizzo,codice_comune)

COMUNE(codice,nome,provincia,regione,numero_abitanti)

FILM(codice,titolo,anno,nazionalita,tecnica_animazione)

PROIETTATO_IN(codice_film,codice_sala,data_ora,lingua)

SALA(codice,nome,capienza,abilitata_3d,codice_cinema)

1. Selezionare tutti i film di tecnica animazione «computer graphics» proiettati nel 2016

Esercizio E.1 Soluzione

CINEMA(codice,nome, indirizzo,codice_comune)

COMUNE(codice,nome,provincia,regione,numero_abitanti)

FILM(codice,titolo,anno,nazionalita,tecnica_animazione)

PROIETTATO_IN(codice_film,codice_sala,data_ora,lingua)

SALA(codice,nome,capienza,abilitata_3d,codice_cinema)

1. Selezionare tutti i film di tecnica animazione «computer graphics» proiettati nel 2016

```
SELECT DISTINCT F.codice, F.titolo  
FROM FILM AS F JOIN PROIETTATO_IN AS P ON F.codice = P.codice_film  
WHERE YEAR(P.data_ora) = 2016 AND tecnica_animazione = 'computer graphics'
```

Esercizio E.2 Soluzione

CINEMA(codice,nome, indirizzo,codice_comune)

COMUNE(codice,nome,provincia,regione,numero_abitanti)

FILM(codice,titolo,anno,nazionalita,tecnica_animazione)

PROIETTATO_IN(codice_film,codice_sala,data_ora,lingua)

SALA(codice,nome,capienza,abilitata_3d,codice_cinema)

2. Selezionare la capienza media delle sale dei cinema che hanno più di una sala

Esercizio E.2 Soluzione

CINEMA(codice,nome, indirizzo,codice_comune)

COMUNE(codice,nome,provincia,regione,numero_abitanti)

FILM(codice,titolo,anno,nazionalita,tecnica_animazione)

PROIETTATO_IN(codice_film,codice_sala,data_ora,lingua)

SALA(codice,nome,capienza,abilitata_3d,codice_cinema)

2. Selezionare la capienza media delle sale dei cinema che hanno più di una sala

```
SELECT codice_cinema AVG(capienza)
FROM SALA
GROUP BY codice_cinema
HAVING COUNT(*) > 1
```

Esercizio E.3 Soluzione

CINEMA(codice,nome, indirizzo,codice_comune)

COMUNE(codice,nome,provincia,regione,numero_abitanti)

FILM(codice,titolo,anno,nazionalita,tecnica_animazione)

PROIETTATO_IN(codice_film,codice_sala,data_ora,lingua)

SALA(codice,nome,capienza,abilitata_3d,codice_cinema)

3. Selezionare i cinema e nome della città che hanno il maggior numero di posti a sedere.

Esercizio E.3 Soluzione

CINEMA(codice,nome, indirizzo,codice_comune)
COMUNE(codice,nome,provincia,regione,numero_abitanti)
FILM(codice,titolo,anno,nazionalita,tecnica_animazione)
PROIETTATO_IN(codice_film,codice_sala,data_ora,lingua)
SALA(codice,nome,capienza,abilitata_3d,codice_cinema)

3. Selezionare i cinema e nome della città che hanno il maggior numero di posti a sedere.

```
CREATE VIEW CAP_CINEMA AS  
SELECT C.codice, C.nome, SUM(S.capienza) AS capienza_totale  
FROM CINEMA AS C JOIN SALA AS S ON S.codice_cinema = C.codice  
GROUP BY C.codice,C.nome
```

```
SELECT C.codice, C.nome, CC.nome, S.capienza_totale  
FROM CINEMA AS C JOIN CAP_CINEMA AS S ON S.codice_cinema = C.codice  
JOIN COMUNE AS CC ON CC.codice = C.codice_comune  
GROUP BY C.codice,C.nome  
HAVING capienza_totale = (SELECT MAX(capienza_totale)  
                          FROM CAP_CINEMA)
```

Esercizio 2

Data la seguente base di dati

Lavoratore (CF, Nome, Cognome, Età)

Città (Nome, NumeroAbitanti)

HaAbitato (CFPersona, Città, DaAnno, AAnno)

HaLavorato (CFPersona, PIVADitta, Città, DaAnno, AAnno)

Ditta (PIVA, Nome, NumeroImpiegati, CapitaleSociale)

HaAvutoSedeIn (PIVADitta, NomeCittà, DaAnno, AAnno)

Interrogazioni SQL:

- 1) Trovare i lavoratori delle aziende milanesi
- 2) Trovare le partite IVA delle ditte che hanno (o hanno avuto) sedi in tutte le città;
- 3) Per ogni ditta trovare il numero degli impiegati che vi hanno lavorato, risiedendo nella stessa città dove aveva sede la ditta
- 4) Trovare le città che hanno o hanno avuto il maggior numero di ditte
- 5) Trovare le città che hanno avuto il maggior numero di ditte con sede nel 2019

Esercizio 2.1 Soluzione

Lavoratore (CF, Nome, Cognome, Età)

Città (Nome, NumeroAbitanti)

HaAbitato (CFPersona, Città, DaAnno, AAnno)

HaLavorato (CFPersona, PIVADitta, Città, DaAnno, AAnno)

Ditta (PIVA, Nome, NumeroImpiegati, CapitaleSociale)

HaAvutoSedeIn (PIVADitta, NomeCittà, DaAnno, AAnno)

1) Trovare i lavoratori delle aziende milanesi

```
SELECT L.Nome, L.Cognome  
FROM Lavoratore AS L JOIN HaLavorato AS H ON H.CFPersona=L.CF  
WHERE H.Città = 'Milano'
```

Esercizio 2.2 Soluzione

Lavoratore (CF, Nome, Cognome, Età)

Città (Nome, NumeroAbitanti)

HaAbitato (CFPersona, Città, DaAnno, AAnno)

HaLavorato (CFPersona, PIVADitta, Città, DaAnno, AAnno)

Ditta (PIVA, Nome, NumeroImpiegati, CapitaleSociale)

HaAvutoSedeIn (PIVADitta, NomeCittà, DaAnno, AAnno)

2) Trovare le partite IVA delle ditte che hanno (o hanno avuto) sedi in tutte le città;

```
SELECT D.PIVADitta
FROM HaAvutoSedeIn AS D
GROUP BY D.PIVADitta
HAVING COUNT(DISTINCT D.NomeCitta)=(SELECT COUNT(*)
                                     FROM Città)
```

Esercizio 2.3 Soluzione

Lavoratore (CF, Nome, Cognome, Età)

Città (Nome, NumeroAbitanti)

HaAbitato (CFPersona, Città, DaAnno, AAnno)

HaLavorato (CFPersona, PIVADitta, Città, DaAnno, AAnno)

Ditta (PIVA, Nome, NumeroImpiegati, CapitaleSociale)

HaAvutoSedeIn (PIVADitta, NomeCittà, DaAnno, AAnno)

3) Per ogni ditta trovare il numero degli impiegati che vi hanno lavorato, risiedendo nella stessa città dove aveva sede la ditta

```
SELECT H.PIVADitta, COUNT(DISTINCT H.CFPersona)
FROM HaLavorato AS H JOIN HaAvutoSedeIn AS S ON S.PIVADitta =
H.PIVADitta JOIN HaAbitato AS A ON A.CFPersona = H.CFPersona
WHERE (A.AAnno >= S.AAnno AND A.DaAnno <= S.AAnno)
      OR (A.AAnno >= S.DaAnno AND A.AAnno <= S.AAnno) AND
      (H.AAnno >= S.AAnno AND H.DaAnno <= S.AAnno)
      OR (H.AAnno >= S.DaAnno AND H.AAnno <= S.AAnno) ...
GROUP BY H.PIVADitta
```

Esercizio 2.4 Soluzione

Lavoratore (CE, Nome, Cognome, Età)

Città (Nome, NumeroAbitanti)

HaAbitato (CFPersona, Città, DaAnno, AAnno)

HaLavorato (CFPersona, PIVADitta, Città, DaAnno, AAnno)

Ditta (PIVA, Nome, NumeroImpiegati, CapitaleSociale)

HaAvutoSedeIn (PIVADitta, NomeCittà, DaAnno, AAnno)

4) Trovare le città che hanno o hanno avuto il maggior numero di ditte

```
CREATE VIEW ContaCitta AS
```

```
SELECT C.Città, COUNT(*) AS NumCitta
```

```
FROM HaAvutoSedeIn AS C
```

```
GROUP BY C.Città
```

```
SELECT C.Città, COUNT(*) Numero_azienze
```

```
FROM HaAvutoSedeIn AS C
```

```
GROUP BY C.Città
```

```
HAVING Numero_azienze = (SELECT MAX(NumCitta) FROM ContaCitta)
```

Esercizio 2.5 Soluzione

Lavoratore (CF, Nome, Cognome, Età)

Città (Nome, NumeroAbitanti)

HaAbitato (CFPersona, Città, DaAnno, AAnno)

HaLavorato (CFPersona, PIVADitta, Città, DaAnno, AAnno)

Ditta (PIVA, Nome, NumeroImpiegati, CapitaleSociale)

HaAvutoSedeIn (PIVADitta, NomeCittà, DaAnno, AAnno)

5) Trovare le città che hanno avuto il maggior numero di ditte con sede nel 2019

CREATE VIEW ContaSedi **AS**

SELECT I.NomeCittà, **COUNT(*)** **AS** NumDitte

FROM HaAvutoSedeIn **AS** I

WHERE (I.DaAnno>=2019 **AND** I.AAnno<=2019)

GROUP BY I.NomeCittà

SELECT C.NomeCittà, NumDitte

FROM ContaSedi **AS** C

WHERE NumDitte >= **ALL**(**SELECT** NumDitte
FROM ContaSedi)

Esercizio 3

Data la seguente base di dati

Concorso(Codice, NumeroProve, Bando)

Bando(Codice, DataPubblicazione, DataScadenza)

Candidato(CF, Cognome, Nome, Indirizzo, Telefono)

Commissario(CF, Cognome, Nome, NumeroTelefono)

Prova(Concorso, Data, PunteggioMassimo)

Svolgimento(DataProva, ConcorsoProva, Candidato, Punteggio)

Partecipazione(Concorso, Candidato, PunteggioTotale, PosizioneInGraduatoria)

Risposta(Bando, Candidato)

Commissione(Concorso, Commissario, Ruolo)

Interrogazioni SQL:

1. Selezionare tutti i concorsi del 2018
2. Selezionare tutti i candidati che hanno fatto concorsi con bando scaduto nel 2018
3. Trovare i commissari dei concorsi con numero di prove pari a 2
4. Trovare i presidenti dei concorsi il cui bando ha una data di pubblicazione posteriore al 22 maggio 2001
5. Dato un concorso, trovare codice fiscale, nome e cognome del secondo in graduatoria

Esercizio 3.1 Soluzione

Concorso(Codice, NumeroProve, Bando)

Bando(Codice, DataPubblicazione, DataScadenza)

Candidato(CF, Cognome, Nome, Indirizzo, Telefono)

Commissario(CF, Cognome, Nome, NumeroTelefono)

Prova(Concorso, Data, PunteggioMassimo)

Svolgimento(DataProva, ConcorsoProva, Candidato, Punteggio)

Partecipazione(Concorso, Candidato, PunteggioTotale, PosizioneInGraduatoria)

Risposta(Bando, Candidato)

Commissione(Concorso, Commissario, Ruolo)

1) Selezionare tutti i concorsi del 2018

SELECT *

FROM Concorso **AS** C **JOIN** Bando **AS** B **ON** B.Codice = C.Bando

WHERE YEAR(B.DataPubblicazione) = 2018 **OR YEAR**(B.DataScadenza) = 2018

Esercizio 3.2 Soluzione

Concorso(Codice, NumeroProve, Bando)

Bando(Codice, DataPubblicazione, DataScadenza)

Candidato(CF, Cognome, Nome, Indirizzo, Telefono)

Commissario(CF, Cognome, Nome, NumeroTelefono)

Prova(Concorso, Data, PunteggioMassimo)

Svolgimento(DataProva, ConcorsoProva, Candidato, Punteggio)

Partecipazione(Concorso, Candidato, PunteggioTotale, PosizioneInGraduatoria)

Risposta(Bando, Candidato)

Commissione(Concorso, Commissario, Ruolo)

2) Selezionare tutti i candidati che hanno fatto concorsi con bando scaduto nel 2018

```
SELECT DISTINCT C.CF, C.Cognome, C.Nome
FROM Candidato AS C JOIN Partecipazione AS P ON P.Candidato = C.CF
      JOIN Concorso AS S ON S.Codice = P.Concorso
      JOIN Bando AS B ON B.Codice = S.Bando
WHERE YEAR(B.DataScadenza) = 2018
```

Esercizio 3.3 Soluzione

Concorso(Codice, NumeroProve, Bando)

Bando(Codice, DataPubblicazione, DataScadenza)

Candidato(CF, Cognome, Nome, Indirizzo, Telefono)

Commissario(CF, Cognome, Nome, NumeroTelefono)

Prova(Concorso, Data, PunteggioMassimo)

Svolgimento(DataProva, ConcorsoProva, Candidato, Punteggio)

Partecipazione(Concorso, Candidato, PunteggioTotale, PosizioneInGraduatoria)

Risposta(Bando, Candidato)

Commissione(Concorso, Commissario, Ruolo)

3) Trovare i commissari dei concorsi con numero di prove pari a 2

```
SELECT C.CF, C.Nome, C.Cognome
FROM Commissario AS C JOIN Commissione AS P ON P.Commissario = C.CF
      JOIN Concorso AS S ON S.Codice = P.Concorso
WHERE S.NumeroProve = 2
```

Esercizio 3.4 Soluzione

Concorso(Codice, NumeroProve, Bando)

Bando(Codice, DataPubblicazione, DataScadenza)

Candidato(CF, Cognome, Nome, Indirizzo, Telefono)

Commissario(CF, Cognome, Nome, NumeroTelefono)

Prova(Concorso, Data, PunteggioMassimo)

Svolgimento(DataProva, ConcorsoProva, Candidato, Punteggio)

Partecipazione(Concorso, Candidato, PunteggioTotale, PosizioneInGraduatoria)

Risposta(Bando, Candidato)

Commissione(Concorso, Commissario, Ruolo)

4) Trovare i presidenti dei concorsi il cui bando ha una data di pubblicazione posteriore al 22 maggio 2001

```
SELECT C.CF, C.Nome, C.Cognome
FROM Commissario AS C JOIN Commissione AS P ON P.Commissario = C.CF
      JOIN Concorso AS S ON S.Codice = P.Concorso
      JOIN Bando AS B ON B.Codice = S.Bando
WHERE P.Ruolo='Presidente' AND B.DataPubblicazione > '22-may-2001'
```

Esercizio 3.2 Soluzione

Concorso(Codice, NumeroProve, Bando)

Bando(Codice, DataPubblicazione, DataScadenza)

Candidato(CF, Cognome, Nome, Indirizzo, Telefono)

Commissario(CF, Cognome, Nome, NumeroTelefono)

Prova(Concorso, Data, PunteggioMassimo)

Svolgimento(DataProva, ConcorsoProva, Candidato, Punteggio)

Partecipazione(Concorso, Candidato, PunteggioTotale, PosizioneInGraduatoria)

Risposta(Bando, Candidato)

Commissione(Concorso, Commissario, Ruolo)

5) Dato il concorso 113, trovare codice fiscale, nome e cognome del secondo in graduatoria

```
SELECT C.CF, C.Nome, C.Cognome
FROM Candidato AS C JOIN Partecipazione AS P ON P.Candidato = C.CF
WHERE P.Concorso = '113' AND P.PosizioneInGraduatoria = 2
```

Esercizio 4

Data la seguente base di dati

Giocatore(Nome,Forza,Energia)

Casella(Nome)

Oggetto(Nome,Valore)

Partite(Numero,GiocatoreDiTurno)

Posizione(Partita,Giocatore,Casella,NumOrdine)

Adiacenza(Partita,Casella1,Casella2)

Possesso(Partita,Giocatore,Oggetto)

Presenza(Partita,Casella,Oggetto)

Interrogazioni SQL:

- 1) Trovare i giocatori che hanno posseduto il maggior numero di oggetti.
- 2) Trovare gli oggetti più posseduti qualsiasi sia la partita
- 3) Selezionare i giocatori che hanno avuto qualsiasi sia la partita l'oggetto 'spada'
- 4) Selezionare casella, contenente oggetti, che ha avuto il maggior numero di permanenza da parte dei giocatori per la partita 10

Esercizio 4.1 Soluzione

Giocatore(Nome,Forza,Energia)

Casella(Nome)

Oggetto(Nome,Valore)

Partite(Numero,GiocatoreDiTurno)

Posizione(Partita,Giocatore,Casella,NumOrdine)

Adiacenza(Partita,Casella1,Casella2)

Possesso(Partita,Giocatore,Oggetto)

Presenza(Partita,Casella,Oggetto)

- 1) Trovare i giocatori che hanno posseduto il maggior numero di oggetti nella partita 20.

```
CREATE VIEW MaggiorPossesso AS
SELECT Giocatore, COUNT(*) NumPoss
FROM Possesso
WHERE Partita = 20
GROUP BY Giocatore
```

```
SELECT S.Giocatore
FROM Possesso AS S
WHERE S.Partita = 20
GROUP BY S.Giocatore
HAVING COUNT(*) = (SELECT MAX(NumPoss) FROM MaggiorPossesso)
```

Esercizio 4.2 Soluzione

Giocatore(Nome,Forza,Energia)

Casella(Nome)

Oggetto(Nome,Valore)

Partite(Numero,GiocatoreDiTurno)

Posizione(Partita,Giocatore,Casella,NumOrdine)

Adiacenza(Partita,Casella1,Casella2)

Possesso(Partita,Giocatore,Oggetto)

Presenza(Partita,Casella,Oggetto)

2) Trovare gli oggetti più posseduti qualsiasi sia la partita

```
CREATE VIEW MaggiorPossesso AS
SELECT Oggetto, COUNT(*) tot_num_poss
FROM Possesso
GROUP BY Oggetto
```

```
SELECT P.Oggetto, COUNT(*) num_poss
FROM Possesso AS P
GROUP BY P.Oggetto
HAVING num_poss = (SELECT MAX(tot_num_poss) FROM MaggiorPossesso)
```

Esercizio 4.3 Soluzione

Giocatore(Nome,Forza,Energia)

Casella(Nome)

Oggetto(Nome,Valore)

Partite(Numero,GiocatoreDiTurno)

Posizione(Partita,Giocatore,Casella,NumOrdine)

Adiacenza(Partita,Casella1,Casella2)

Possesso(Partita,Giocatore,Oggetto)

Presenza(Partita,Casella,Oggetto)

3) Selezionare i giocatori che hanno avuto qualsiasi sia la partita l'oggetto 'spada'

```
SELECT COUNT(DISTINCT P.Giocatore) AS giocatori_con_spada
FROM Possesso AS P
WHERE P.Oggetto = 'spada'
GROUP BY P. Giocatore
```

Esercizio 4.4 Soluzione

Giocatore(Nome,Forza,Energia)

Casella(Nome)

Oggetto(Nome,Valore)

Partite(Numero,GiocatoreDiTurno)

Posizione(Partita,Giocatore,Casella,NumOrdine)

Adiacenza(Partita,Casella1,Casella2)

Possesso(Partita,Giocatore,Oggetto)

Presenza(Partita,Casella,Oggetto)

4) Selezionare casella, contenente oggetti, che ha avuto il maggior numero di permanenza da parte dei giocatori per la partita 10

```
CREATE VIEW MaggiorPermanenza AS
SELECT P.Casella, COUNT(*) TotPerm
FROM Posizione AS P JOIN Presenza AS Z ON Z.Casella = P.Casella
WHERE P.Oggetto <> NULL AND Z.Partita = 10
GROUP BY P.Casella
```

```
SELECT P.Casella
FROM Posizione AS P JOIN Presenza AS Z ON Z.Casella=P.Casella
WHERE P.Oggetto <> NULL AND Z.Partita = 10
GROUP BY P.Casella
HAVING COUNT(*)= (SELECT MAX(TotPerm) FROM MaggiorPermanenza)
```

Esercizio 3

Si consideri il seguente schema di basi di dati che descrive le prenotazioni on-line di una compagnia di traghetti nazionali che effettua diverse tratte. La compagnia dispone di più navi. Ogni cliente al primo accesso deve registrarsi ed ottiene il proprio codice, valido per gli accessi successivi. Effettuando la prenotazione il cliente indica il tipo di sistemazione (cabina, poltrona, passaggio ponte, ...) ed in caso di trasporto di auto ne indica la targa.

CLIENTE (codice_cliente, nome, cognome, telefono, primo_accesso, ultimo_accesso)
PRENOTAZIONE(codice_prenotazione, codice_cliente, portopartenza, portoarrivo, nome_nave, data_prenotazione, data_partenza, numero_passeggeri, sistemazione, auto, costo)
NAVE (nome_nave, anno_immatricolazione, numero_cabine)
TRATTE(portopartenza, portoarrivo, distanza)

Rispondere alle seguenti interrogazioni in SQL:

1. Trovare i clienti (codice_cliente, nome e cognome) che si sono registrati ma non hanno mai effettuato prenotazioni
2. Trovare i clienti (codice_cliente, nome e cognome) che hanno effettuato nel 2009 un viaggio di andata e ritorno.
3. Trovare la nave che ha trasportato più passeggeri nel 2009

Esercizio 3.1 Soluzione

CLIENTE (codice_cliente, nome, cognome, telefono, primo_accesso, ultimo_accesso)

PRENOTAZIONE(codice_prenotazione, codice_cliente, portopartenza, portoarrivo, nome_nave,
data_prenotazione, data_partenza, numero_passeggeri, sistemazione, auto, costo)

NAVE (nome_nave, anno_immatricolazione, numero_cabine)

TRATTE(portopartenza, portoarrivo, distanza)

1. Trovare i clienti (codice_cliente, nome e cognome) che si sono registrati ma non hanno mai effettuato prenotazioni.

Esercizio 3.1 Soluzione

CLIENTE (codice_cliente, nome, cognome, telefono, primo_accesso, ultimo_accesso)

PRENOTAZIONE(codice_prenotazione, codice_cliente, portopartenza, portoarrivo, nome_nave,
data_prenotazione, data_partenza, numero_passeggeri, sistemazione, auto, costo)

NAVE (nome_nave, anno_immatricolazione, numero_cabine)

TRATTE(portopartenza, portoarrivo, distanza)

1. Trovare i clienti (codice_cliente, nome e cognome) che si sono registrati ma non hanno mai effettuato prenotazioni.

```
SELECT codice_cliente, nome, cognome  
FROM CLIENTE  
WHERE codice_cliente NOT IN (SELECT codice_cliente  
                             FROM PRENOTAZIONE)
```

Esercizio 3.2 Soluzione

CLIENTE (codice_cliente, nome, cognome, telefono, primo_accesso, ultimo_accesso)

PRENOTAZIONE(codice_prenotazione, codice_cliente, portopartenza, portoarrivo, nome_nave,
data_prenotazione, data_partenza, numero_passeggeri, sistemazione, auto, costo)

NAVE (nome_nave, anno_immatricolazione, numero_cabine)

TRATTE(portopartenza, portoarrivo, distanza)

2. Trovare i clienti (codice_cliente, nome e cognome) che hanno effettuato nel 2009 un viaggio di andata e ritorno.

Esercizio 3.3 Soluzione

CLIENTE (codice_cliente, nome, cognome, telefono, primo_accesso, ultimo_accesso)
PRENOTAZIONE(codice_prenotazione, codice_cliente, portopartenza, portoarrivo, nome_nave,
data_prenotazione, data_partenza, numero_passeggeri, sistemazione, auto, costo)
NAVE (nome_nave, anno_immatricolazione, numero_cabine)
TRATTE(portopartenza, portoarrivo, distanza)

3. Trovare la nave che ha trasportato più passeggeri nel 2009

```
CREATE VIEW navepasseggeri2009 (nave, somma_passeggeri) AS  
SELECT nome_nave, SUM(numero_passeggeri)  
FROM PRENOTAZIONE  
WHERE data_partenza <= 31-12-2009 AND data_partenza >= 1-1-2009  
GROUP BY nome_nave
```

```
SELECT nave  
FROM navepasseggeri2009  
WHERE somma_passeggeri >= (SELECT MAX(somma_passeggeri)  
FROM navepasseggeri2009)
```

Esercizio 4

Data la seguente base di dati:

AUTORE (id, Nome, Cognome, Ruolo, dipartimento)

PUBBLICAZIONE (autore, articolo, EprimoAutore)

ARTICOLO (id, titolo, nomeRivista, NumeroPubblicazione, PaginaInizio, PaginaFine, annoPubblicazione, NumeroCitazioni, IFRivista)

Dove:

- Ruolo rappresenta il ruolo dell'autore all'interno de dipartimento e può assumere valori come Prof. Ordinario, Prof Associato, Ricercatore, Assegnista, Phd ...
- IFRivista rappresenta l'impact factor della rivista caratterizzato da un intero positivo maggiore di 0
- L'attributo EprimoAutore è booleano.

Scrivere in SQL le seguenti interrogazioni

1. Si riportino tutti gli articoli (incluso il titolo e il nome della rivista in cui è stato pubblicato) in cui Andrea Maurino è primo autore
2. Si riporti il numero degli articoli pubblicati su riviste con IF >20 in cui almeno un professore ordinario è stato autore
3. Si trovi il titolo dell'articolo che ha avuto più citazioni nel 2015

Esercizio 4.1 Soluzione

AUTORE (id, Nome, Cognome, Ruolo, dipartimento)

PUBBLICAZIONE (autore, articolo, EprimoAutore)

ARTICOLO (id, titolo, nomeRivista, NumeroPubblicazione, PaginaInizio, PaginaFine, annoPubblicazione, NumeroCitazioni, IFRivista)

1. Si riportino tutti gli articoli (incluso il titolo e il nome della rivista in cui è stato pubblicato) in cui Andrea Maurino è primo autore

```
SELECT P.titolo, P.nomeRivista
```

```
FROM (AUTORE AS A JOIN PUBBLICAZIONE AS Pub
```

```
ON A.id = Pub.autore) JOIN ARTICOLO AS P ON (Pub.articolo = P.id)
```

```
WHERE A.Nome = "Andrea" AND A.Cognome = "Maurino"
```

```
AND Pub.EprimoAutore = "Yes"
```

Esercizio 4.2 Soluzione

AUTORE (id, Nome, Cognome, Ruolo, dipartimento)

PUBBLICAZIONE (autore, articolo, EprimoAutore)

ARTICOLO (id, titolo, nomeRivista, NumeroPubblicazione, PaginaInizio, PaginaFine, annoPubblicazione, NumeroCitazioni, IFRivista)

2. Si riporti il numero degli articoli pubblicati su riviste con IF >20 in cui almeno un professore ordinario è stato autore

```
SELECT COUNT(DISTINCT Pub.articolo)
```

```
FROM (AUTORE AS A JOIN PUBBLICAZIONE AS Pub ON
```

```
A.id = Pub.autore) JOIN ARTICOLO AS P ON Pub.articolo = P.id
```

```
WHERE A.Ruolo = "Professore ordinario") AND P.IFRivista > 20
```

//senza il distinct un paper scritto da due ordinari è contato due volte

Esercizio 4.3 Soluzione

AUTORE (id, Nome, Cognome, Ruolo, dipartimento)

PUBBLICAZIONE (autore, articolo, EprimoAutore)

ARTICOLO (id, titolo, nomeRivista, NumeroPubblicazione, PaginaInizio, PaginaFine, annoPubblicazione, NumeroCitazioni, IFRivista)

3. Si trovi il titolo dell'articolo che ha avuto più citazioni nel 2015

```
SELECT titolo
```

```
FROM ARTICOLO
```

```
WHERE annoPubblicazione=2015 AND NumeroCitazioni =
```

```
(SELECT MAX(citazioni)
```

```
FROM ARTICOLO
```

```
WHERE annoPubblicazione=2015)
```

//È possibile farlo anche con una query nidificata, o con una vista

Esercizio 5

Data la seguente base di dati relativa alle comande, ovvero alle ordinazioni, di un ristorante. Si assuma che una volta che il cliente ha saldato il conto tutte le informazioni relative alle ordinazioni di quel tavolo sono cancellate.

TAVOLO (numeroTavolo, numeroCoperti, Luogo)

ORDINAZIONE (IdTavolo, idPiatto, quantità)

PIATTO (idPiatto, NomePiatto, Categoria, Descrizione, PrezzoUnitario)

CONTIENE (idPiatto, idAllergene)

ALLERGENI (idAllergene, nomeAllergene)

Dove:

- I valori ammissibili di Categoria sono: Antipasti, Primi, Secondi, Contorni, Frutta, Dessert, Altro.
- Si noti come un piatto può contenere 0,1 o più allergeni.

Scrivere in SQL le seguenti interrogazioni

- 1) Trovare il numero del tavolo che ha ordinato almeno un “Contorno” con prezzo superiore a 7 euro
- 2) Trovare i piatti che contengono l’allergene “latte” ma non “frutta a guscio”
- 3) Trovare il tavolo con almeno 5 coperti che ha il maggior numero di ordinazioni

Esercizio 5.1 Soluzione

TAVOLO (numeroTavolo, numeroCoperti, Luogo)

ORDINAZIONE (IdTavolo, idPiatto, quantità)

PIATTO (idPiatto, NomePiatto, Categoria, Descrizione, PrezzoUnitario)

CONTIENE (idPiatto, idAllergene)

ALLERGENI (idAllergene, nomeAllergene)

- 1) Trovare il numero del tavolo che ha ordinato almeno un “Contorno” con prezzo superiore a 7 euro

Esercizio 5.1 Soluzione

TAVOLO (numeroTavolo, numeroCoperti, Luogo)

ORDINAZIONE (IdTavolo, idPiatto, quantità)

PIATTO (idPiatto, NomePiatto, Categoria, Descrizione, PrezzoUnitario)

CONTIENE (idPiatto, idAllergene)

ALLERGENI (idAllergene, nomeAllergene)

- 1) Trovare il numero del tavolo che ha ordinato almeno un “Contorno” con prezzo superiore a 7 euro

```
SELECT DISTINCT IdTavolo  
FROM ORDINAZIONE AS O JOIN PIATTO AS P ON (O.idPiatto=P.idpiatto)  
WHERE P.categoria = “Contorno” AND P.prezzoUnitario > 7
```

Esercizio 5.2 Soluzione

TAVOLO (numeroTavolo, numeroCoperti, Luogo)

ORDINAZIONE (IdTavolo, idPiatto, quantità)

PIATTO (idPiatto, NomePiatto, Categoria, Descrizione, PrezzoUnitario)

CONTIENE (idPiatto, idAllergene)

ALLERGENI (idAllergene, nomeAllergene)

2) Trovare i piatti (id e nome) che contengono l'allergene "latte" ma non "frutta a guscio"

Esercizio 5.2 Soluzione

TAVOLO (numeroTavolo, numeroCoperti, Luogo)

ORDINAZIONE (IdTavolo, idPiatto, quantità)

PIATTO (idPiatto, NomePiatto, Categoria, Descrizione, PrezzoUnitario)

CONTIENE (idPiatto, idAllergene)

ALLERGENI (idAllergene, nomeAllergene)

2) Trovare i piatti (id e nome) che contengono l'allergene "latte" ma non "frutta a guscio"

```
SELECT P.idPiatto, P.NomePiatto
FROM PIATTO AS P JOIN CONTIENE AS C ON (P.idPiatto = C.idpiatto)
      JOIN ALLERGENI AS A ON (C.idAllergene = A.idAllergene)
WHERE A.NomeAllergene = "latte"
```

EXCEPT

```
SELECT P.idPiatto, P.NomePiatto
FROM PIATTO AS P JOIN CONTIENE AS C ON (P.idPiatto = C.idpiatto)
      JOIN ALLERGENI AS A ON (C.idAllergene = A.idAllergene)
WHERE A.NomeAllergene = "Frutta a Guscio"
```

Esercizio 5.2 Soluzione

TAVOLO (numeroTavolo, numeroCoperti, Luogo)

ORDINAZIONE (IdTavolo, idPiatto, quantità)

PIATTO (idPiatto, NomePiatto, Categoria, Descrizione, PrezzoUnitario)

CONTIENE (idPiatto, idAllergene)

ALLERGENI (idAllergene, nomeAllergene)

2) Trovare i piatti (id e nome) che contengono l'allergene "latte" ma non "frutta a guscio"

```
SELECT P.idPiatto, P.NomePiatto
FROM PIATTO AS P JOIN CONTIENE AS C ON (P.idPiatto = C.idpiatto)
    JOIN ALLERGENI AS A ON (C.idAllergene = A.idAllergene)
WHERE A.NomeAllergene = "latte" AND P.idPiatto NOT IN
    (SELECT C1.idPiatto
    FROM CONTIENE AS C1 JOIN ALLERGENI AS A1 ON
        (C1.idAllergene = A1.idAllergene)
    WHERE A1.NomeAllergene = "Frutta a Guscio"
    )
```

Esercizio 5.3 Soluzione

TAVOLO (numeroTavolo, numeroCoperti, Luogo)

ORDINAZIONE (IdTavolo, idPiatto, quantità)

PIATTO (idPiatto, NomePiatto, Categoria, Descrizione, PrezzoUnitario)

CONTIENE (idPiatto, idAllergene)

ALLERGENI (idAllergene, nomeAllergene)

3) Trovare il tavolo con almeno 5 coperti che ha il maggior numero di ordinazioni

Esercizio 5.3 Soluzione

TAVOLO (numeroTavolo, numeroCoperti, Luogo)

ORDINAZIONE (IdTavolo, idPiatto, quantità)

PIATTO (idPiatto, NomePiatto, Categoria, Descrizione, PrezzoUnitario)

CONTIENE (idPiatto, idAllergene)

ALLERGENI (idAllergene, nomeAllergene)

3) Trovare il tavolo con almeno 5 coperti che ha il maggior numero di ordinazioni

```
SELECT O.idTavolo, COUNT(*) AS NumeroOrdinazioni
FROM ORDINAZIONE AS O JOIN TAVOLO AS T
      ON (T.numeroTavolo = O.idTavolo)
WHERE T.numeroCoperti >= 5
GROUP BY O.idTavolo
HAVING COUNT(*) >= ALL (SELECT COUNT(*)
                        FROM ORDINAZIONE AS O JOIN TAVOLO AS T
                        ON (T.numeroTavolo = O.idTavolo)
                        WHERE T.numeroCoperti >= 5
                        GROUP BY O.idTavolo)
```

Esercizio 3

Si consideri la seguente Base di Dati:

DISCO(Nroserie, TitoloAlbum, Anno, Prezzo)
CONTIENE(NroSerieDisco, CodiceReg, NroProgr)
ESECUZIONE(CodiceReg, TitoloCanz, Anno)
AUTORE(Nome, TitoloCanzone)
CANTANTE(NomeCantante, CodiceReg)

Si formulino le seguenti query in SQL

1. La canzone che è stata eseguita almeno due volte
2. Gli autori di canzoni eseguite tra il 2000 e il 2010
3. Gli autori delle canzoni più eseguite

Esercizio 3.1 Soluzione

DISCO(Nroserie, TitoloAlbum, Anno, Prezzo)
CONTIENE(NroSerieDisco, CodiceReg, NroProgr)
ESECUZIONE(CodiceReg, TitoloCanz, Anno)
AUTORE(Nome, TitoloCanzone)
CANTANTE(NomeCantante, CodiceReg)

1. La canzone che è stata eseguita almeno due volte

Esercizio 3.1 Soluzione

DISCO(Nroserie, TitoloAlbum, Anno, Prezzo)
CONTIENE(NroSerieDisco, CodiceReg, NroProgr)
ESECUZIONE(CodiceReg, TitoloCanz, Anno)
AUTORE(Nome, TitoloCanzone)
CANTANTE(NomeCantante, CodiceReg)

1. La canzone che è stata eseguita almeno due volte

```
SELECT E.TitoloCanz, COUNT(*)  
FROM ESECUZIONE AS E  
GROUP BY E.TitoloCanz  
HAVING COUNT(*) > 1
```

Esercizio 3.2 Soluzione

DISCO(Nroserie, TitoloAlbum, Anno, Prezzo)
CONTIENE(NroSerieDisco, CodiceReg, NroProgr)
ESECUZIONE(CodiceReg, TitoloCanz, Anno)
AUTORE(Nome, TitoloCanzone)
CANTANTE(NomeCantante, CodiceReg)

2. Gli autori di canzoni eseguite tra il 2000 e il 2010

Esercizio 3.2 Soluzione

DISCO(Nroserie, TitoloAlbum, Anno, Prezzo)
CONTIENE(NroSerieDisco, CodiceReg, NroProgr)
ESECUZIONE(CodiceReg, TitoloCanz, Anno)
AUTORE(Nome, TitoloCanzone)
CANTANTE(NomeCantante, CodiceReg)

2. Gli autori di canzoni eseguite tra il 2000 e il 2010

```
SELECT DISTINCT A.nome  
FROM Autore AS A JOIN ESECUZIONE AS E ON E.TitoloCanz = A.TitoloCanzone  
WHERE E.anno >=2000 AND E.anno <= 2010
```

Esercizio 3.3 Soluzione

DISCO(Nroserie, TitoloAlbum, Anno, Prezzo)
CONTIENE(NroSerieDisco, CodiceReg, NroProgr)
ESECUZIONE(CodiceReg, TitoloCanz, Anno)
AUTORE(Nome, TitoloCanzone)
CANTANTE(NomeCantante, CodiceReg)

3. Gli autori delle canzoni più eseguite

Esercizio 3.3 Soluzione

DISCO(Nroserie, TitoloAlbum, Anno, Prezzo)
CONTIENE(NroSerieDisco, CodiceReg, NroProgr)
ESECUZIONE(CodiceReg, TitoloCanz, Anno)
AUTORE(Nome, TitoloCanzone)
CANTANTE(NomeCantante, CodiceReg)

3. Gli autori delle canzoni più eseguite

```
SELECT A.Nome  
FROM AUTORE AS A  
WHERE A.TitoloCanzone IN (  
    SELECT E.TitoloCanz  
    FROM ESECUZIONE AS E  
    GROUP BY E.TitoloCanz  
    HAVING COUNT(*) >= ALL(  
        SELECT COUNT (*)  
        FROM ESECUZIONE AS E  
        GROUP BY E.TitoloCanz))
```

